

VAPT Audit Engine - Architecture, Implementations & Technical Fixes

1. Executive Summary

This document provides a comprehensive technical overview of the Vulnerability Assessment & Penetration Testing (VAPT) Audit Engine built into the AICyberAuditBox platform.

The system ingests raw vulnerability scan outputs (Nessus XML/HTML, Qualys CSV, Burp Suite, Nmap), dynamically extracts all ground-truth security findings (e.g., 122 vulnerabilities across target hosts), maps them to 15 Master VAPT Controls (`VAPT-1` to `VAPT-15`), and produces executive-ready PDF & DOCX compliance reports.

2. VAPT Module Architecture

The VAPT module provides an end-to-end automated workflow:

`Upload raw tool reports -> Parse & normalize findings -> RAG Vector Retrieval & LangGraph AI Assessment -> Inline Auditor Review -> Validate Ground Truth -> Generate Executive PDF/DOCX Reports`.

```text

Tool Report File (Nessus .nessus/.html / CSV / Nmap XML / BurpSuite XML / Scope .xlsx)

?

v

[Frontend: Streamlit app.py - File Uploader & Sidebar Framework Controls]

?

v

[Parsers: nessus\_parser.py / control\_mapper.py - Ingestion & Normalization]

??? parse\_nessus\_xml() / parse\_html() Vulnerability & Target IP Extraction

??? map\_finding\_to\_control() Dynamic Categorization to VAPT-1 .. VAPT-15

??? \_enrich\_finding\_metadata() CVSS v4.0 Severity Scoring & Risk Mapping

?

v

[Database (SQLite): audit\_db.db - AuditReport / AuditCheckpoint / Findings]

?

v

[RAG Engine & LangGraph Agent: audit\_graph.py + nomic-embed-text + bge-reranker-base]

??? Vector Embedding & Retrieval nomic-embed-text (768-dim Vector Space)

??? Cross-Encoder Reranking bge-reranker-base (Context Relevance > 0.75)

??? LLM Inference Node llama-server / llama.cpp (gemma-4b, n\_predict=1024)

??? 4-Gate Grounding Validation Grounding, Evidence Schema & Hallucination Check

?

v

[Frontend: Streamlit Auditor Workspace - Interactive Review & Inline Edit]

??? Accept / Modify / Reject Controls & Auditor Commenting

??? Proof of Concept & OCR Provenance Viewer

??? Pill Badges: Verified Vulnerability Proof (Amber) vs Verified Compliant (Green)

?

v

[Validation Pass: report\_exporter.py - validate\_and\_derive\_report\_payload()]

??? Host Target IPs Extraction Extract real scanned host IPs from findings

??? CVSS v4.0 Metric Derivation Calculate vectors & scores dynamically per finding

??? Policy & Evidence Strict Rule Require BOTH Policy & Evidence for Compliant status

?

v

[Export Engine: report\_exporter.py - Dual PDF & DOCX Generator]

??? PDF Exporter (FPDF2): Cover page + Scope IPs + CVSS grid + Executive Summary

??? DOCX Exporter (python-docx): Per-finding detail cards + Remediation steps

?

v

[Browser: Direct Streamlit Download -> Executive PDF (.pdf) & Word (.docx)]

```

3. Master VAPT Framework Control Mapping (`VAPT-1` to `VAPT-15`)

Raw vulnerability findings are automatically classified into 15 Master Technical Control Objectives:

Control ID	Control Name	Scope & Finding Classifications
---	---	---
VAPT-1	External Network Penetration Test	Perimeter open ports, exposed services
VAPT-2	Reconnaissance & OSINT	Public footprinting, DNS leaks, exposed emails
VAPT-3	Host & Port Enumeration	Open RPC, SMB, NetBIOS, Telnet services
VAPT-4	Web Application Security	XSS, SQLi, CSRF, missing HTTP headers
VAPT-5	Internal Network Penetration Test	Internal RCEs, directory traversal (e.g., WinRAR CVEs)
VAPT-6	Active Directory & Domain Security	Kerberoasting, AS-REP roasting, LDAP signing
VAPT-7	Privilege Escalation & Exploit Testing	UAC bypass, local root/system exploits (e.g., Amazon SSM)
VAPT-8	Email & Social Engineering Defense	Missing SPF/DKIM/DMARC email security
VAPT-9	Wireless Security	WPA2 Enterprise, rogue APs
VAPT-10	API & Cloud Security	Unauthenticated REST/JWT endpoints, Swagger leaks
VAPT-11	EDR & AV Evasion	Missing or disabled Endpoint Protection
VAPT-12	Vulnerability Assessment & Patching	Outdated/End-of-Life software (ASP.NET EOL, .NET Core EOL)
VAPT-13	Network Segmentation	Unrestricted subnet routing, missing firewalls
VAPT-14	Secure Configuration Baseline	Weak TLS ciphers, default credentials, FileZilla flaws
VAPT-15	Post-Exploitation & Remediation	Patch verification and remediation re-tests

4. Key Technical Implementations & Architectural Enhancements

4.1. Structural Field Derivation & Ground-Truth Validation

- Implementation: Added `validate\_and\_derive\_report\_payload()` in `src/ui/report\_exporter.py`.
- Impact: Eliminates hardcoded template fallbacks. Every output field (Target IPs, CVSS Vectors, Remediation Text, Testing Dates) is derived directly from input scan files before report compilation.

4.2. Dynamic Scope Target Host IP Extraction

- Implementation: Dynamically parses host IPs from Nessus/Qualys finding target fields (`target`, `host`, `ip`) and scan headers.
- Impact: Replaced static sample IP fallbacks with real scanned target hosts (`13.126.199.93`, `3.108.211.52`, etc.).

4.3. CVSS v4.0 Vector & Score Derivation per Control

- Implementation: Dynamically calculates CVSS v4.0 base metric vectors and severity scores (`10.0 Critical`, `9.8 High`, `7.5 Medium`, `5.3 Low`) for each specific vulnerability.
- Impact: Every finding card and report table displays unique, accurate CVSS scores rather than static repeating numbers.

4.4. Strict 2-Way Audit Compliance Rule

- Implementation: Enforced strict audit rule in `\_enrich\_finding\_metadata()` in `src/ui/app.py`.
- Impact: A control status is ONLY `COMPLIANT` overall if BOTH Policy AND Technical Evidence are present & compliant. If Policy doc is missing (`Policy doc missing`), overall status is set to `NON-COMPLIANT`.

4.5. Visual Pill Badge Clarity (Amber vs. Green)

- Implementation: Updated evidence badge rendering in `app.py`.
- Impact:
 - Compliant Controls: Displays `? Verified Compliant Evidence` in Green (`#22c55e`).
 - Non-Compliant Findings with Proof: Displays `? Verified Vulnerability Proof` in Amber/Orange (`#97316`), eliminating green badge contradictions on non-compliant cards.

4.6. Elimination of Synthetic Revision Paper Trails

- Implementation: Updated PDF exporter to omit the Revision History table entirely unless the user explicitly enters real team review dates in the UI.
- Impact: Prevents auto-generating synthetic or fake review dates.

4.7. Custom Auditor & Client Branding Support

- Implementation: Full sidebar custom inputs for Auditor Firm Name, Lead Auditors, Reviewers, Approvers, Document ID, Target

Client Organization, Client Contact Details, and Custom Logo uploads.

- Impact: Fully customizes both PDF and DOCX exports for any auditing organization.

4.8. Performance & Timeout Optimization

- Implementation: Capped `n\_predict: 1024` tokens, added Gemma stop tokens (`<end\_of\_turn>`, `<eos>`), and capped reflection retries to 1 pass in `src/ai/audit\_graph.py`.

- Impact: Reduced full VAPT audit execution times from 10+ minutes down to ~1.5 - 2 minutes total (~15s per control).

5. How to Run & Verify

1. Pull the latest code on Azure VM:

```
```powershell
git pull origin vapt
```
```

2. Launch the server:

```
```powershell
.\run_llamacpp_demo.bat
```
```

3. Upload scan file (e.g., `NOCPL\_yu0k9r.html`), select VAPT, run audit, and click Export PDF / DOCX!